

08

Logistic Regression

Predicting YES or NO — the simplest classifier

Don't let the name fool you: this is for categories, not numbers

AI/ML Engineer Learning Series · Deep Dive Edition

What it does	Predicts a category — usually yes/no, 0/1
Type	Supervised learning — classification
The trick	Bends the line into an S-shape (0 to 1)
Output	A probability, like 'this is 80% spam'
Why learn it	The go-to first classifier, base of neural networks

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is Logistic Regression?	Why the name is misleading
2	The Problem with a Straight Line	Why we need a new shape
3	The Sigmoid (S-curve)	The heart of the whole model
4	From Probability to Decision	The threshold idea
5	How It Learns	Cost function and training
6	Doing It in Code	Full sklearn example
7	Reading the Output	Probabilities and predictions
8	Measuring Success	Confusion matrix, precision, recall
9	More Than Two Classes	Multi-class classification
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is Logistic Regression?

Logistic Regression (■■■■■■■■■■ ■■■■■■■■■■) has a confusing name. It says 'regression' (predicting a number), but it actually does **classification** (predicting a category). It is the simplest and most popular model for yes/no questions.

It answers questions like:

- Is this email spam or not spam?
- Will this customer buy or not buy?
- Is this tumour harmful or harmless?
- Did the passenger survive or not? (Titanic)

Instead of giving a hard yes/no straight away, it first gives a **probability** (■■■■■■■■■■) — a number between 0 and 1. For example '0.85' means '85% chance this is spam'. Then you turn that probability into a decision.

■ *Remember the difference: Linear Regression predicts a NUMBER (price, marks). Logistic Regression predicts a CATEGORY (yes/no). Same family, different job.*

2. Why a Straight Line Doesn't Work

Our answers here are only 0 (no) or 1 (yes). If we try to fit a straight line (like in linear regression), it goes wrong fast. A straight line keeps rising forever — it can predict 1.5 or -0.3, which make no sense as a probability.

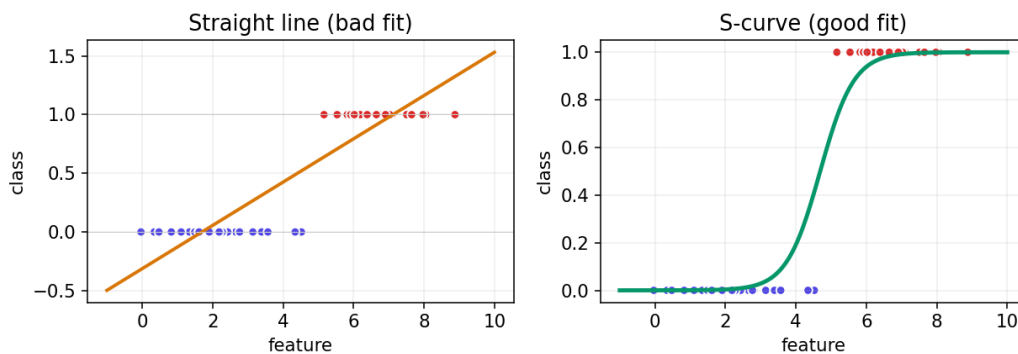


Fig 1 — Left: a straight line goes above 1 and below 0 (nonsense). Right: the S-curve stays neatly between 0 and 1.

We need a shape that **always stays between 0 and 1**, is low on the left, high on the right, and bends smoothly in the middle. That special shape is the sigmoid.

3. The Sigmoid — The S-Curve

The **sigmoid** (■■■■■■■■■■) is a function shaped like the letter S. You can feed it any number — huge, tiny, negative — and it always squashes the result into a value between 0 and 1. That's exactly what we want for a probability.

$$\text{sigmoid}(z) = 1 / (1 + e^{-z})$$

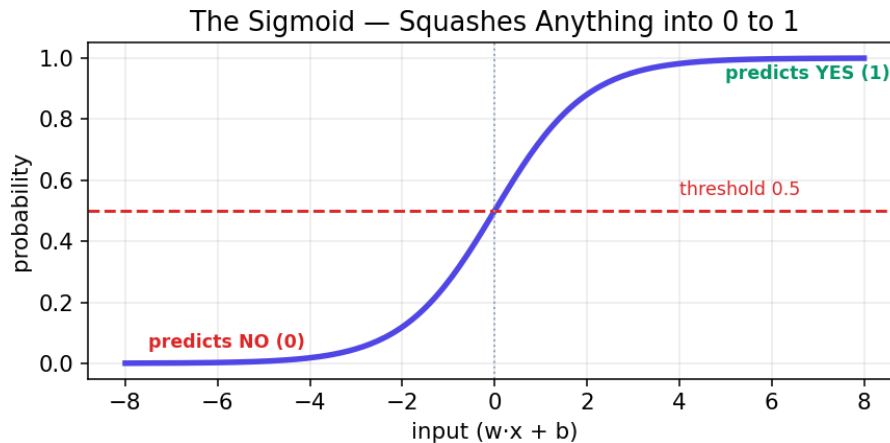


Fig 2 — The sigmoid. Far left → near 0 (no). Far right → near 1 (yes). Middle → smooth bend.

How the model works, in two simple steps:

- **Step 1:** compute a straight-line score, just like before: $z = w \cdot x + b$
- **Step 2:** pass that score through the sigmoid to turn it into a probability.

So logistic regression is really linear regression with a sigmoid squashing the output. The straight-line part decides the score; the sigmoid turns it into a clean 0-to-1 probability.

4. From Probability to a Decision

The sigmoid gives a probability, but eventually you need a real answer: yes or no. You make that choice using a **threshold** (■■■■■■■■■■ — ■■■■■■■■■■), usually 0.5:

- If probability $\geq 0.5 \rightarrow$ predict **YES (1)**
- If probability $< 0.5 \rightarrow$ predict **NO (0)**

You can move the threshold to suit your problem. For something serious like disease detection, you might lower it to 0.3 — better to catch more possible cases even with some false alarms. For something where false alarms are costly, you might raise it to 0.7.

■ *The threshold is a choice YOU make, not something the model decides. Changing it trades off catching more cases vs making fewer false alarms.*

5. How It Learns

Just like linear regression, logistic regression learns by finding the best w and b . But it uses a different cost function called **log loss** (or cross-entropy). You don't need the heavy math — just the idea:

- If the model is confident and correct → very small penalty.
- If the model is confident but wrong → very big penalty.
- Gradient descent then adjusts w and b to make the total penalty as small as possible.

Why not just use MSE like before? Because with the sigmoid, MSE creates a bumpy cost landscape that's hard to roll down. Log loss gives a smooth bowl shape that gradient descent can handle nicely — so training works well.

6. Doing It in Code

Same friendly fit/predict pattern. Only the model name changes:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create and train
model = LogisticRegression()
model.fit(X_train, y_train)

# Hard predictions (0 or 1)
preds = model.predict(X_test)

# Soft predictions (probabilities)
probs = model.predict_proba(X_test)
# each row: [prob of class 0, prob of class 1]

print('Accuracy:', accuracy_score(y_test, preds))
print(classification_report(y_test, preds))
```

■ *predict() gives the final 0/1 answer. predict_proba() gives the raw probabilities — useful when you want to set your own threshold or rank predictions by confidence.*

7. Reading the Output

After training, here's what you can pull out:

Output	What it means
model.predict(X)	Final answers: 0 or 1 for each row
model.predict_proba(X)	Probability for each class (0 to 1)
model.coef_	Weights — how each feature pushes the answer
model.intercept_	The bias b

Understanding weights here: a positive weight means that feature pushes the prediction toward YES. A negative weight pushes toward NO. The bigger the weight, the stronger the push. This makes logistic regression easy to explain.


```
# sklearn handles multi-class automatically
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train) # y has 3+ classes? no problem

model.predict(X_test)      # picks the most likely class
model.predict_proba(X_test) # probability for EACH class
```

■ If you ever see a warning about 'max_iter', it just means training needed more steps. Add max_iter=1000 to give it more time to settle.

10. Common Mistakes

Mistake	Why it's a problem	Fix
Only checking accuracy	Hides failures on rare classes	Use precision, recall, F1
Forgetting to scale	Slow or shaky training	Scale features first
Ignoring class imbalance	Model favours the big class	Use class_weight='balanced'
Always using 0.5 threshold	May not fit your goal	Tune the threshold
Expecting a number output	It predicts categories	Use linear regression for numbers

Quick Reference Cheatsheet

Task	Code
Import	<code>from sklearn.linear_model import LogisticRegression</code>
Create model	<code>model = LogisticRegression(max_iter=1000)</code>
Train	<code>model.fit(X_train, y_train)</code>
Predict class	<code>model.predict(X_test)</code>
Predict probability	<code>model.predict_proba(X_test)</code>
See weights	<code>model.coef_</code>
Accuracy	<code>accuracy_score(y_test, preds)</code>
Full report	<code>classification_report(y_test, preds)</code>
Confusion matrix	<code>confusion_matrix(y_test, preds)</code>
Precision	<code>precision_score(y_test, preds)</code>
Recall	<code>recall_score(y_test, preds)</code>
F1-score	<code>f1_score(y_test, preds)</code>
Handle imbalance	<code>LogisticRegression(class_weight='balanced')</code>

Key idea to remember:

$$\text{score} = w \cdot x + b \rightarrow \text{sigmoid} \rightarrow \text{probability (0 to 1)}$$

Next Topic: Evaluation Metrics (Accuracy, Precision, Recall, F1, Support) — a full deep dive into exactly how we judge whether a model is actually good.